Muhammad Ammar Bin Talib

BSCS 23143
Section – B
PDC Assignment 4

StudySync Architectural Analysis & Design

PART 1: Root Cause Analysis

Bug1 - Synchronization: The Lost Update Anomaly
The current architecture implements document persistence as a naive **read-modify-write cycle** with no transaction isolation or concurrency control. When User A and User B both issue a GET request to the same document endpoint concurrently, the FastAPI server executes two independent `SELECT` queries and returns identical in-memory snapshots to both clients. Each user then makes their local edits and issues a PUT request. The server processes each PUT as an *unconditional* `UPDATE documents SET content = :new WHERE id = :id` with no check on whether the row has been modified since the read. Whichever write arrives second silently overwrites the first. This is the canonical **Lost Update anomaly** which is a well-documented concurrency hazard that arises whenever two transactions read the same data item and then update it based on the value they read, without the second writer knowing the first has already committed. The root cause is the total absence of a versioning predicate or exclusive lock between the SELECT and the UPDATE at both the database and API lifecycle layers.

Bug2 - Coordination: Dropped Webhook and Permanent State Divergence
The application relies on Clerk to emit a webhook HTTP POST to a FastAPI endpoint whenever a user cancels their subscription. The naive handler processes this event synchronously which results in updating the local database record and returning a 200 response. This design has two critical gaps. First, Clerk's webhook delivery is **best-effort, not guaranteed**: a transient network blip, a DNS failure, or a server crash occurring between receiving the event and committing the database transaction will silently swallow the event. Second, because the handler stores no **idempotency key** (the Clerk Event ID), any redelivery attempt would double-process the event, risking data corruption. The consequence is a **permanent state divergence**: Clerk's authoritative record marks the user as cancelled, while the local database still grants premium access (a split-brain condition) that no internal process will ever automatically detect or heal.

Bug3 - Fault Tolerance: Synchronous LLM Call as a Cascading Single Point of Failure
The LLM summarisation feature invokes the external API via a direct, synchronous `requests.post()` call inside a FastAPI route handler. Uvicorn's default ASGI worker pool allocates a finite number of OS threads for I/O-bound work. When the upstream LLM service degrades and begins hanging (taking 30-60 seconds before timing out) each blocked request holds a worker thread for the entire duration. As new requests arrive, they each claim an additional thread. Once the thread pool is exhausted, the Uvicorn process stops accepting new connections entirely. Critically, this failure is *not scoped to the LLM feature* as it propagates across the entire application, preventing authentication, document saves, and all other endpoints from being served. This is a textbook **cascading failure** caused by the absence of

any timeout enforcement, circuit breaker, or asynchronous offloading around the external dependency. The LLM API is a hard **single point of failure** for the whole system.

PART 2 — Architectural Design & Proposed Solutions

Fix1 - Optimistic Locking with a Version Column (Synchronization)
Add an integer `version` column to the `documents` table (default 1). Every GET response includes the current version as a client-held token. Every PUT request must supply that token. The server executes a *conditional update*:

```
UPDATE documents SET content = :new_content, version = version + 1
WHERE id = :doc_id AND version =
:expected_version; -- optimistic predicate
```

If `rows_affected == 0`, the row's version has already advanced meaning a concurrent writer committed first. The server returns **HTTP 409 Conflict**, and the client must re-fetch the latest version before retrying. This requires *no database locks*, adds negligible latency under low contention, and guarantees linearisable writes under high concurrency.
Sequence Diagram, Optimistic Locking: Two Concurrent Writers
Green rows = successful commit. Red rows = rejected stale write. Both users start at version 1; only the first committer succeeds.

#	User A	FastAPI Server	Database	User B
1	GET /doc -->	SELECT … WHERE id=1 -->		
2	<-- {content, v=1}	<-- {id=1, version=1}		GET /doc -->
3				<-- {content, v=1}
4	PUT /doc {edit_A, v=1} -->	UPDATE … WHERE v=1 SET v=2 -->		
5			<-- rows_affected=1 OK	
6	<-- 200 OK {v=2}			
7				PUT /doc {edit_B, v=1} -->
8		UPDATE … WHERE v=1 SET v=2 -->		
9			<-- rows_affected=0 STALE	
10				<-- 409 Conflict "Reload & retry"
	Lost Update PREVENTED — User B must re-fetch version 2 before retrying.			

Fix2 - Idempotent Webhook Handler with Dead-Letter Queue (Coordination)
Introduce a `processed_events` table with a unique constraint on the **Clerk Event ID** (idempotency key). On each webhook arrival: (1) if the Event ID already exists, return 200 immediately without re-processing; (2) otherwise open a database transaction, persist the subscription change, and insert the Event ID atomically resulting in making the handler **exactly-once safe**. Decouple processing entirely by publishing the raw payload to an **async task queue** (Redis + Celery). A background worker retries with **exponential back-off** (2^n s, capped at 15 min); permanently failed events are forwarded to a **Dead-Letter Queue (DLQ)** for manual inspection. No cancellation can be silently dropped.

Fix3 - Circuit Breaker with Graceful Fallback (Fault Tolerance)

Wrap every LLM call in a **Circuit Breaker**. In **CLOSED** state, calls proceed normally; consecutive failures increment a counter. At the threshold (e.g., 3 failures) the breaker trips to **OPEN**: all calls are *short-circuited* in microseconds, returning a graceful fallback (*"AI tutor temporarily unavailable"*) without touching the network and eliminating thread blocking entirely. After a recovery timeout the breaker enters **HALF-OPEN**, allows one probe, and resets to CLOSED on success or back to OPEN on failure. The LLM outage is strictly contained; all other endpoints remain fully operational.

CAP Theorem Trade-off Analysis

The Optimistic Locking fix is a deliberate **CP** design: it prioritises **Consistency over Availability**, rejecting the conflicting write rather than allowing silent corruption. The losing writer suffers brief unavailability (a 409 and retry), but data integrity is preserved which is the correct trade-off for a collaborative editor. The Circuit Breaker makes the opposite choice which is explicitly **AP**: during an LLM partition the system stays available by serving a degraded fallback, accepting that the response is not authoritative. This suits a supplementary feature where a graceful degradation beats a full outage. The Webhook DLQ is also **CP**: subscription-state correctness is non-negotiable, so the system accepts processing latency (queued retries) rather than risk permanently inconsistent billing records.

Together, these choices reflect intentional, feature-level CAP positioning rather than a single uniform system stance.